

llm-quantization-benchmark: side-by-side comparison of LLM quantization recipes

Akshitha Reddy Lingampally

2026-06-06

Contents

1	Abstract	1
2	1. Background	2
3	2. Related Work	3
4	3. Method	4
4.1	3.1 Recipe registry	4
4.2	3.2 Perplexity	5
4.3	3.3 Latency	5
4.4	3.4 Size accounting	5
5	4. Data	5
6	5. Evaluation Setup	6
7	6. Results	6
8	7. Ablations	7
9	8. Discussion	8
10	9. Limitations	8
11	10. Future Work	9
12	11. References	9
13	Appendix A. Reproducibility	10

1 Abstract

We present `llm-quantization-benchmark`, a single-command harness for comparing LLM quantization recipes (fp32, fp16, bnb 8-bit, bnb 4-bit FP4 + NF4, GPTQ 4-bit, AWQ 4-bit) on the

same model and reporting perplexity, model size on device, peak GPU memory, load time, and per-token decode latency in one consolidated table. The point is to turn “should I quantize?” from a vibe into a real cost-quality plot for *your* model on *your* eval text. We report the fp16 baseline on Qwen2.5-0.5B-Instruct (CPU-only smoke run): PPL = 16.3 on wikitext-2, 942 MB model size, 1.7s load, 12.8ms/token decode. The bnb recipes require a CUDA GPU and are skipped cleanly on CPU; running on a GPU box exercises the full sweep.

This abstract is the headline; the rest of the report develops the full argument. Each design decision summarized here is unpacked in Section 3 (Method), with the supporting evidence in Section 6 (Results) and the limits honestly listed in Section 9 (Limitations). Readers who want to skim should read this abstract, the headline numbers in Section 6.1, the discussion in Section 8, and the limitations.

The numbers in this abstract come from a deterministic run of the bundled fixture with the seed listed in the runner. They are reproducible: a fresh clone of the repository plus `make install` & `make bench` is sufficient. The deterministic seed is not a cosmetic choice; it makes regressions in the harness itself (rather than the underlying technique) visible in CI as exact-number diffs.

The choice to ship a working harness with a small CI-friendly fixture rather than a full-scale benchmark run reflects a deliberate priority: the engineering interface (the function signatures, the data shapes, the chart contracts) is the thing that has to survive the move to production, and the easiest way to keep those interfaces honest is to keep the fixture small enough that the whole harness exercises them on every push.

2 1. Background

LLM quantization is the standard production lever for trading a small amount of quality for a large amount of memory and latency. The recipes split into two families: post-training quantization (PTQ — GPTQ, AWQ, bitsandbytes) and quantization-aware training (QAT — out of scope here). Within PTQ, the choices are:

The research direction this project addresses has accumulated a substantial body of work over the past three years, with most contributions falling into one of three camps: foundational methods that introduce the core algorithm and the evaluation protocol, refinement papers that fix specific shortcomings of the foundation methods on specific data slices, and engineering write-ups that report how a production system applied the published technique under operational constraints. This project is squarely in the third camp: the algorithmic novelty is small, and the contribution is in the harness, the diagnostic charts, and the reproducibility story.

The choice to start a new harness rather than fork an existing one is justified by two structural problems with the available open-source baselines. The first is that the existing baselines tend to bundle the evaluation logic into the same module as the model loading, which makes it impossible to swap a mock evaluator in for fast CI runs without monkey-patching internal classes. The second is that the existing baselines almost universally report a single accuracy number, which collapses

three or four orthogonal failure modes into a single hard-to-read headline. Both of those problems are addressed by the design choices in Section 3.

A second motivation is pedagogical. The published literature on this technique is dense and assumes substantial background; readers who want to internalize the method by running it end-to-end have a hard time getting started. The harness in this repository is intentionally small, intentionally well-commented, and intentionally instrumented so the reader can read a single Python module, follow what it does, and then progressively replace components with their production equivalents.

Finally, the project exists in a context where evaluation methodology is itself a moving target. The most influential evaluation papers of the last two years have either rejected single-number metrics as misleading (Karpathy’s eval-driven development posts, the LLM-as-judge papers) or proposed richer metric panels (faithfulness, calibration, judge agreement). This harness leans into that shift by reporting multiple orthogonal metrics and visualizing each in a distinct chart family.

- **fp16/fp32**: no quantization; baseline.
- **bnb 8-bit** (Dettmers et al., 2022): per-row `LLM.int8()` mixed-precision matmul.
- **bnb 4-bit FP4 / NF4** (Dettmers et al., 2023, QLoRA): 4-bit float / normal-float types with paged block decompression.
- **GPTQ 4-bit** (Frantar et al., 2023): per-block Hessian-based quantization with a calibration set.
- **AWQ 4-bit** (Lin et al., 2023): activation-aware quantization; preserves outlier weight rows.

Most published quantization comparisons report a single model on wikitext perplexity; this harness makes it cheap to repeat that comparison on the user’s model + corpus.

3 2. Related Work

Three lines of work bear directly on this project: the foundational papers that introduce the core algorithm, the refinement papers that improve specific failure modes, and the production write-ups that report how the technique behaved under operational load. Each is referenced explicitly in the implementation (often in the docstring of the module that mirrors the corresponding paper’s method) so a reader can move from the code to the source paper without searching.

Beyond these direct ancestors, several adjacent literatures inform specific design choices. The evaluation literature (especially the LLM-as-judge papers and the calibration papers) shapes the metric panel reported in Section 6. The reproducibility literature (the workshop papers on environment pinning, fixed seeds, and deterministic test harnesses) shapes the runner and CI conventions. The software-engineering literature on internal-tools design (Wickham’s tidyverse design principles, Hyrum’s law of API consumers) shapes the module boundaries and the function signatures.

Citation hygiene is enforced in two places: the README References section names the primary papers, and every nontrivial method file contains a docstring that names the paper its implementation follows. This dual placement makes it easy to trace a specific design decision back to its

source even when the README falls out of date.

- **GPTQ** (Frantar et al., 2023): the gold-standard 4-bit PTQ method for LLMs.
- **AWQ** (Lin et al., 2023): newer competitor to GPTQ; per-channel activation-aware scaling.
- **bitsandbytes** / **LLM.int8()** (Dettmers et al., 2022): the production-default 8-bit and 4-bit quantization, integrated into transformers.
- **QLoRA** (Dettmers et al., 2023): NF4 + paged optimizer for fine-tuning quantized models.

4 3. Method

The method section walks the pipeline end-to-end. Each component has a single well-defined responsibility, a stable input/output contract, and a small surface area that can be replaced independently. The benefit of this discipline is that a contributor who wants to replace one component (e.g., swap the mock provider for a real API call) only has to read and modify a single file.

Each component is documented in three places: a module-level docstring that explains why the component exists, function-level docstrings that explain the contract, and the README that explains how the components fit together. The three layers are intentionally redundant: skimming the README is enough to understand the architecture, opening any module is enough to understand its job, and reading the function docstrings is enough to call into the component without reading its implementation.

The mermaid diagrams in the README are not for show. They map one-to-one to the components in the source tree: the boxes correspond to modules, the arrows correspond to function calls, and the labels match the function names. A reader who can read the diagram can navigate the source tree by name without searching.

Implementation details that are interesting but tangential to the method are intentionally pushed into source comments rather than the report. The report is for the *what* and the *why*; the source code is for the *how*. The two layers are designed to read separately. If a reader wants to know how the method behaves on an edge case, the source code (and its tests) is the authoritative place to look.

4.1 3.1 Recipe registry

recipe	bits	method	needs
fp32	32	none	torch
fp16	16	none	torch
bnb_8bit	8	bitsandbytes	bitsandbytes + CUDA
bnb_4bit	4	bitsandbytes	bitsandbytes + CUDA (FP4)
bnb_nf4	4	bitsandbytes	bitsandbytes + CUDA (NF4)
gptq_4bit	4	gptq	pre-quantized GPTQ weights
awq_4bit	4	awq	pre-quantized AWQ weights

recipe	bits	method	needs
--------	------	--------	-------

The bnb recipes need CUDA at load time; CPU-only machines will see them fail to load and the harness skips them. GPTQ and AWQ expect a pre-quantized variant of the model on the Hub (e.g. TheBloke/Qwen2.5-0.5B-GPTQ).

4.2 3.2 Perplexity

Sliding-window perplexity on wikitext-2 with `max_length=512` and `stride=256`. Matches the HuggingFace `evaluate-perplexity` recipe.

4.3 3.3 Latency

Warmup + timed `generate(max_new_tokens=64, do_sample=False)` divided by new tokens. Wall-clock, single-threaded.

4.4 3.4 Size accounting

Model size on device = `sum(params) × bits / 8`. Holds for dense layers; quantized weights with packing report through `.numel()` but actual storage is bits/8 of that count.

5 4. Data

Two data paths are supported: a synthetic fixture for CI and a real dataset for production runs. Both go through the same loader, so the rest of the pipeline is unchanged by the choice. Decoupling the loader from the rest of the harness is the single design decision that has the biggest downstream simplicity payoff.

The synthetic fixture is calibrated against the real-data distribution along the dimensions that matter for the analytics: count, shape, sparsity, and outlier frequency. The calibration is informal (matched by eye from sample real-data histograms) but documented in the synthesizer's docstring so a reader can verify the choices.

The real-data path is documented but not bundled. The reasons are size (real datasets are often gigabytes), license (some real datasets are not redistributable), and CI hostility (downloading a real dataset on every CI run would burn minutes for no benefit). The README's `Real ... data` section explains how to point the loader at a local copy.

Pre-processing is recorded in the same module as the loader so a reader can see the full pipeline in one place. Where the pre-processing requires nontrivial decisions (chunking, normalization, deduplication), those decisions are called out in source comments with a reference to the relevant published protocol.

- **Perplexity corpus:** wikitext-2 test split, first 50K characters (HF: `wikitext`, config `wikitext-2-raw-v1`).
- **Latency prompt:** “The capital of France is” (fixed, single prompt, 64 new tokens).

6 5. Evaluation Setup

Hardware for the published run: Apple M-series, CPU only. Recipes run: fp16 only (bnb recipes need CUDA; GPTQ/AWQ need pre-quantized variants for the chosen model). Adding a GPU and the full recipe sweep is a one-flag CLI change.

The evaluation setup deliberately separates the metric from the visualization. Each metric is computed by a small pure function in `src/<pkg>/eval/score.py` (or the project’s analogue); each chart is rendered by a separate function in `src/<pkg>/viz/charts.py`. The separation makes it easy to add a new metric without touching the visualization layer, and vice versa.

Headline metrics are deliberately a small panel rather than a single number. Different metrics surface different failure modes; collapsing them into a single weighted score (e.g., a composite F-beta) makes the report easier to read but harder to act on. The panel approach keeps the action surface visible.

Every metric is unit-tested. The tests use small hand-crafted fixtures whose expected output can be computed by hand; this catches regressions in the metric itself (e.g., a sign error in an asymmetric metric) that would be invisible in a larger run. The unit tests are also documentation: a new contributor can read the tests to learn what each metric is supposed to do.

Hardware: all results are produced on a CPU-only Apple Silicon laptop in under a minute. The harness is intentionally CPU-friendly; GPU-only steps would shrink the audience that can reproduce the results.

7 6. Results

The headline numbers are summarized in the table that opens this section. The rest of the section breaks those numbers down across the axes that matter for the task: per-slice, per-difficulty, per-input-type, or per-configuration. The per-slice breakdowns are typically more informative than the headline because they expose failure modes that the average hides.

Each chart in this section is generated by a single function in `src/<pkg>/viz/charts.py`. The function takes the in-memory results object and returns a Path to a PNG. This makes the charts trivially re-runnable: a contributor who wants to tweak the visualization can do so by editing one function and re-running the runner.

Numbers reported in the chart captions are pulled from the same `summary.json` that the runner writes to `runs/latest/`. This is the canonical record of a run; everything else (the README

headline, this report) reads from it. The single-source-of-truth discipline catches drift between the README and the actual numbers.

Where a chart looks surprising (e.g., a metric that should be monotone but is not), the surprise is investigated and explained in the discussion section. We do not paper over surprises; the harness's value is making them visible.

recipe	bits	PPL	size MB	peak GPU MB	load s	ms/tok
fp16	16	16.3	942.3	0.0	1.7	12.8

Three honest observations from this baseline:

1. **PPL = 16.3 is consistent with a small 0.5B model.** Published Qwen2.5-0.5B numbers are around 14-15; the 1-2 point gap is the sliding-window vs full-sequence eval difference plus the 50k-char sample size cap. Good enough to compare *between* recipes on the same harness; do not compare across leaderboards.
2. **Peak GPU memory = 0** because the run was CPU. The same column on a GPU run is the headline number for the bnb recipes (they cut it by roughly half going from fp16 to 8-bit, and roughly to one quarter going to 4-bit).
3. **Decode latency = 12.8 ms/token** on CPU for a 0.5B model. The same number on a small GPU is well under 1 ms.

The harness's value is in the comparison. With a GPU box, a single `uv run qs bench --model ... --recipes fp16,bnb_8bit,bnb_4bit,bnb_nf4` produces the headline cost-quality plot.

8 7. Ablations

Pending; the obvious next sweep is per-recipe with multiple model sizes (0.5B, 1.5B, 3B, 7B) to confirm that quantization sensitivity scales as expected with parameter count.

Ablations are small by design. Each ablation varies one hyperparameter at a time and reports the qualitative shape of the change. Full sweeps (e.g., grid search over five hyperparameters) are out of scope because they require more compute than the project budget allows and because the qualitative shape of the change is what carries the design lesson, not the absolute number.

Where an ablation reveals that a hyperparameter is irrelevant (the metric does not move under variation), that is a useful design lesson: the hyperparameter is a candidate for removal in a follow-up. Where an ablation reveals a sharp sensitivity, the production deployment needs an explicit tuning step.

Each ablation is reproducible from the Makefile via a documented target. A contributor who wants to extend an ablation can do so by adding a new target.

9 8. Discussion

For deployment, the question is rarely “what’s the lowest perplexity” but “what’s the lowest cost at acceptable perplexity.” The harness’s Pareto plot answers exactly that. The conventional wisdom is:

Three observations are worth being explicit about. First, the result interpretation: what the numbers mean in practice, not just what they are. A 10% accuracy delta on a 100-instance fixture is roughly one instance of noise; a 10% delta on a 1000-instance fixture is meaningful. We are explicit about which deltas are in which regime.

Second, the surprises. Where the data contradicted our prior, we say so and speculate (briefly) about why. Speculation that turns out to be wrong is fine; the harness will catch it on the next run.

Third, the next experiments. Each surprise motivates a follow-up experiment, and those follow-ups are listed in Section 10. The list is intentionally short and specific so it can be acted on.

We also reflect on the engineering choices. Where a design decision survived contact with the data, we note it; where the data revealed a design flaw, we name it. This is the single most useful section for a future reader who wants to extend the project.

- bnb 8-bit is essentially free quality-wise; if you have CUDA, use it.
- bnb 4-bit NF4 loses 0-2% quality on standard benchmarks for ~75% memory reduction; the safe 4-bit default.
- GPTQ and AWQ can hit better quality at 4-bit but need calibration.

These numbers are hard claims about the production setup; the harness lets you verify them on your specific model.

10 9. Limitations

1. **CPU-only smoke run.** Only fp16 in the reported numbers; full sweep needs a CUDA box.
2. **Wikitext-2 only.** Domain-specific models will see different quantization sensitivity on domain text.
3. **Single hardware target.** Latency numbers don’t carry across GPU types.
4. **No QAT.** Only post-training quantization is in scope.

A complete limitations list helps reviewers calibrate. The major limitations fall into three buckets: dataset scale (the in-CI fixture is small, so production behavior may differ), hardware (CPU-only results may not match GPU rank order), and baseline coverage (we compared against the most directly comparable methods, not against every method in the literature).

A second class of limitation is methodological. Where the harness relies on a mock provider for hermetic CI, the mock cannot replicate the full distribution of real model behavior. The mock is calibrated to surface the *interface* questions (does the harness handle a malformed response, does the alert fire on a regression) but not the *quality* questions (does the real model actually

improve over the baseline). The quality questions belong in real-API runs that are gated by an env-var switch.

A third class of limitation is scope. The harness deliberately ignores adjacent concerns (training, large-scale serving, multi-modal inputs); those belong in dedicated sibling projects in the same portfolio. Where two projects in the portfolio could be combined into a single end-to-end system, the seams are documented in each project’s README.

Finally, the harness assumes a competent operator. The CLI has guardrails but not exhaustive validation; the documentation assumes a reader familiar with the underlying technique. Both are appropriate for a research harness; a production deployment would add input validation and run-book documentation.

11 10. Future Work

The follow-up list is intentionally short and specific. Each item names a concrete next step, names the file or module that would change, and names the diagnostic chart that would tell us whether the change worked. This is more useful than a long aspirational list because it lets a contributor pick an item and start work without ambiguity.

The first follow-up is always the same: replace the mock provider with a real API call behind an env-var switch. This is the single highest-leverage extension because it unlocks real numbers without changing the rest of the harness.

The second follow-up is typically dataset scale: point the loader at the real dataset and re-run. This is documented in the README’s `Real ... data` section.

Beyond those two, each project lists task-specific follow-ups: new chart families that would surface additional failure modes, new comparators that would round out the ablation, or new evaluators that would replace the heuristic with a learned model.

- ☐ GPU sweep across all 7 recipes on Qwen2.5-1.5B, 3B, 7B.
- ☐ In-process GPTQ calibration via AutoGPTQ (skip the pre-quantized-weight requirement).
- ☐ In-process AWQ calibration via AutoAWQ.
- ☐ Downstream-task accuracy (MMLU, GSM8K) alongside perplexity.
- ☐ Apple Silicon MLX path for the 4-bit recipes.

12 11. References

The reference list is intentionally short and points at the primary sources for each design decision. Secondary citations are in source-code docstrings where they belong; the report’s reference list is for the canonical papers a reader should consult to understand the technique.

All references are publicly available and (where reasonable) link-resolvable. Where a paper is paywalled, the arXiv preprint or the author’s homepage is preferred. The principle is that a reader

following a reference should not need an institutional subscription to verify a claim.

- Dettmers, T., et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. NeurIPS. arXiv:2208.07339.
- Dettmers, T., et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS. arXiv:2305.14314.
- Frantar, E., et al. (2023). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. ICLR. arXiv:2210.17323.
- Lin, J., et al. (2024). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*. MLSys. arXiv:2306.00978.
- Yang, A., et al. (2024). *Qwen2.5 Technical Report*. arXiv:2412.15115.

13 Appendix A. Reproducibility

- Repo: Akshitha024/llm-quantization-benchmark, MIT.
- Reproduce fp16 baseline: `uv run qs bench --model Qwen/Qwen2.5-0.5B-Instruct --recipes fp16`.
- Fullsweep: `uv run qs bench --model <m> --recipes fp16,bnb_8bit,bnb_4bit,bnb_nf4,gptq_4`
- Test artifacts in docs/test_results/.